

Twenty Years of Bigtable

Fabio Baltieri
Bora Beran
Igor Bernstein
Aimiee Borda
Adrian Chan
Mark D'Andrea
Artak Dashyan
Ramesh Dharan
Gabor Dinnyes
Mike Dominguez
Eric Dorland
Jose Duenas
Gary Elliott
Bruno Furtado
Madison Garcia
Marçal Garolera Huguet
Brendan Gleason
Alexis Hawkins
Jeremy Hurwitz
Google
Sunnyvale, California, USA

Anoshak Irani
Rohit Jog
Sudarshan Kadambi
Vikram Khemka
Sailesh Krishnamurthy
Maxim Krivokon
Bruce Lee
Xiaozhou Li
Tom Magrino
Matt Maly
Mark Mangrich
Douglas McErlean
Pablo Montes
Li Moore
Eduardo Morales
Greg Morris
An Nguyen
Steve Niemitz
Gaurav Prabhu Gaonkar
Google
Sunnyvale, California, USA

Jim Rutherford
Stephen Ryan
Sho Saha
Kanoj Sarcar
Cristina Schmidt
Andrii Shyshkalov
Pratibha Suryadevara
Nick Suttle
Anvit Tawar
John Tobin
Justin Uang
Phaneendhar Vemuru
Harendra Verma
Shitanshu Verma
Jinghang Wang
Michal Wegorek
Siu Yau
Andrius Ziukas
Google
Sunnyvale, California, USA

Abstract

Bigtable is a pioneering and influential non-relational database system. The original Bigtable paper [17] has been widely cited and it inspired and influenced many other systems such as HBase and Cassandra. Since then, Bigtable has continued to grow and has become one of the largest database systems inside Google. In this paper, we tell the journey of Bigtable inside Google for the last twenty years. We present new features added and improvements made to Bigtable, and we share our experience of running this storage system at scale, continually improving all aspects to accommodate the ever-growing demands of users.

CCS Concepts

• **Information systems** → **Database design and models; Information storage systems.**

Keywords

Bigtable; non-relational databases; wide-column stores

ACM Reference Format:

Fabio Baltieri, Bora Beran, Igor Bernstein, Aimiee Borda, Adrian Chan, Mark D'Andrea, Artak Dashyan, Ramesh Dharan, Gabor Dinnyes, Mike

Dominguez, Eric Dorland, Jose Duenas, Gary Elliott, Bruno Furtado, Madison Garcia, Marçal Garolera Huguet, Brendan Gleason, Alexis Hawkins, Jeremy Hurwitz, Anoshak Irani, Rohit Jog, Sudarshan Kadambi, Vikram Khemka, Sailesh Krishnamurthy, Maxim Krivokon, Bruce Lee, Xiaozhou Li, Tom Magrino, Matt Maly, Mark Mangrich, Douglas McErlean, Pablo Montes, Li Moore, Eduardo Morales, Greg Morris, An Nguyen, Steve Niemitz, Gaurav Prabhu Gaonkar, Jim Rutherford, Stephen Ryan, Sho Saha, Kanoj Sarcar, Cristina Schmidt, Andrii Shyshkalov, Pratibha Suryadevara, Nick Suttle, Anvit Tawar, John Tobin, Justin Uang, Phaneendhar Vemuru, Harendra Verma, Shitanshu Verma, Jinghang Wang, Michal Wegorek, Siu Yau, and Andrius Ziukas. 2026. Twenty Years of Bigtable. In *Companion of the International Conference on Management of Data (SIGMOD Companion '26)*, May 31–June 05, 2026, Bengaluru, India. ACM, New York, NY, USA, 13 pages. <https://doi.org/10.1145/3788853.3803095>

1 Introduction

Bigtable is a pioneering and influential non-relational database system built by Google around 2004. The Bigtable paper [17] was published in OSDI 2006 and has garnered over 8000 citations [39]. Furthermore, Bigtable helped co-usher the non-relational movement and inspired other systems such as HBase [30] and Cassandra [33].

In this paper, we tell the journey of Bigtable over the last twenty years since that first paper, detailing the new features and improvements we have made for Bigtable. Today, Bigtable is used by thousands of teams and users in virtually every Product Area (PA) inside Google. At the time of this writing, Bigtable manages 10 EB (exabyte) of data [3] and has a peak total QPS (query per second)



This work is licensed under a Creative Commons Attribution 4.0 International License. *SIGMOD Companion '26, Bengaluru, India*
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2450-3/2026/05
<https://doi.org/10.1145/3788853.3803095>

of 7 billion [4]. There are more than 1.6 quadrillion (10^{15}) rows in a single table, more than 1 EB in a single table, and more than 250 million QPS in a single cluster.

Bigtable's core strengths reside in its simple architecture, highly flexible storage format, and its focus on performance. These strengths have endured through the years and have remained a pivotal part of the system's identity. Virtually every evolution of the system can be tied back to these principles, from serving as the experimental foundation for a whole generation of systems in Google, to new features such as replication, counters, materialized views, et cetera. For example, many of these newer features all depend on the same idea of asynchronous processing that is not on the foreground write path. Likewise, flexible schemas have also continued to be key to the system. Furthermore, while Bigtable was originally designed with scale in mind, continuous demand, coupled with the ever-widening variety of use cases required that we improve Bigtable across several dimensions. However, the original design persists and has continued to scale. Bigtable's overall architecture remains largely the same as originally envisioned, attesting to the enduring value of an extendable architecture.

In this paper, we present several developments in Bigtable since the original paper (new features, and improvements on key aspects), and we share our operational experience of running Bigtable across two decades. Due to space limitations, we can only highlight a subset of the work, and for the work presented, we only go into a moderate level of technical details.

For new features, we first present Bigtable replication, arguably the single most important feature because it enables Bigtable to be globally available, a requirement for many use cases. We present how Bigtable replication works under the hood, especially how multi-primary replication works (namely, how multiple replicas can accept writes from clients and then propagate the writes to other replicas).

Google launched its cloud product in 2008 [49]. As part of Google Cloud, we launched Cloud Bigtable in 2015 [18] and this service is a very popular part of the Cloud Database portfolio. Cloud customers bring in many interesting use cases, feature requests, performance and reliability requirements that we had not yet encountered internally at Google and have required specialized treatments. It is worth noting that cloud customers run on the exact same backend as internal customers, although there are differences in control plane, connectivity, and client architectures. We will present a few new features that were first requested by Google Cloud customers, such as SQL support, Change Data Capture (CDC) [7], Counters and CRDTs (conflict-free replicated data types) [42], and Materialized Views (MV) [8].

We then discuss how Bigtable has continually improved on several key aspects of a distributed storage system, including scalability, performance, reliability, and resource efficiency. Scalability has been a key design principle of Bigtable since day one, and the continuous demand in the intervening twenty years has required us to constantly improve Bigtable's scalability using a wide variety of techniques. Likewise, we have made significant improvements to Bigtable's performance, especially read performance. For a storage system managing 10EB of data, reliability is paramount; we have developed a number of techniques to ensure Bigtable data is reliable. We then present how we continuously improved Bigtable's

resource efficiency, as compute resources continue to be in high demand and short supply.

Shortly after Bigtable went into production, we quickly learned most teams didn't have the expertise to run it well. In response, we shifted to operating Bigtable under a service model, in which the Bigtable SRE team runs the cluster. This model has proven successful: for almost twenty years, Bigtable has been primarily run as a service inside Google. The last part of the paper shares our operational experience.

The rest of this paper is organized as follows. Section 2 gives a brief overview of the key concepts in Bigtable. Section 3 describes replication in Bigtable. Section 4 presents recent new features on analytics. Section 5, Section 6, Section 7, and Section 8 highlight how we improved Bigtable's scalability, performance, reliability, and resource efficiency, respectively. Section 9 shares some operational experience we learned in the last twenty years. Section 10 discusses related work. Section 11 concludes the paper.

2 Background

This section gives a brief overview of the basic concepts in Bigtable relevant to the present paper. For further details, please refer to the original Bigtable paper [17].

Data in Bigtable is stored in *tables*, which are a three-dimensional sorted map. The map is indexed by a *row key*, a *column key*, and a *timestamp*, and each *value* in this map is an uninterpreted array of bytes, namely: $(\text{row_key}:\text{string}, \text{column_key}:\text{string}, \text{timestamp}:\text{int64}) \rightarrow \text{value}:\text{bytes}$. Bigtable organizes a table's data in lexicographic order by the row key, and divides the row space into contiguous segments called *tablets*, which are the unit of hosting (by a tablet server), loading, unloading, movement, and load balancing. Column keys are grouped into sets called *column families*, and a column key is structured as *column_family:column_name*. Column families are further grouped into *locality groups*, where each locality group forms the unit of storage, e.g., in compactions, a separate SSTable (Sorted String Table) [29] is generated for each locality group in each tablet.

To persist data, Bigtable combines a write-ahead log (WAL, called a *commit log*) [34] and a log-structured merge (LSM) tree [36]. In particular, all mutations are logged to a commit log before confirming the mutation to the caller. Recent mutations are stored in an in-memory data structure called the *memtable*. Bigtable manages persisted data using three primary types of compactions: minor, merge, and major. A *minor compaction* writes the memtable to an SSTable when it becomes full. Over time, a growing stack of SSTables are formed by minor compactions. When the stack becomes too deep, it can impede the performance of various operations (e.g., reads become slow because now more SSTables have to be searched). High stack depth triggers a *merge compaction*, which merges the top K (chosen by the merge compaction algorithm) SSTables into one SSTable. Lastly, a *major compaction* merges all the SSTables in the stack into one SSTable, performing data deletion and garbage collection in the process.

A *Bigtable cluster* consists of a master and a (potentially large) number of *tablet servers*. (Note: the name master was chosen over 20 years ago and we wouldn't choose it today. For consistency with

the original paper [17] we will continue using master in this paper.) A row-column-timestamp intersection is called a *data cell*. The tablet servers host the tablets and the master decides which tablet(s) should be hosted by which tablet server. Tablet servers are grouped into *partitions* and those in the same partition are further grouped into *subpartitions*. A location lookup protocol enables a client to locate which tablet server is hosting the data (tablets) needed. Clients fetch data directly from the tablet servers. Depending on the load on each tablet server, tablets are moved around servers to achieve load balancing. Unlike some other non-relational systems, the data in tablets is stored in a distributed file system (called Colossus [26]) rather than on the server's disk, so tablet moves are relatively cheap - data does not need to be moved. Tablets are auto-merged or split depending on load to avoid hotspotting. All metadata is internally hosted in Bigtable (namely, stored in a table in Bigtable), except for the bootstrap metadata needed to find the first metadata tablet, which is stored in Chubby [14].

3 Replication

Replication has been the single biggest new feature in Bigtable since the original paper. Today, most Bigtable users make use of replication to ensure data availability and reliability. It is not uncommon for users to replicate data across dozens of replicas, and our largest replication user has over 200 replicas.

3.1 Concepts

Bigtable replication mirrors *column families* from one table to another, either in the same cluster or across data centers. One can use these replicated families to provide local access to data for improved read/write performance, to handle failover in case one table goes down, or as a data backup mechanism. A table with replicated families is called a *replica*. A set of replicas that mirror data with each other is called a *replication configuration*. A replica does not in any way coordinate the admission of the mutation with other replicas, thereby providing high availability and low cost.

Bigtable provides *eventual consistency*, meaning that the writes done to a replica will *eventually* be propagated to other replicas, and users can query how up-to-date the data in each replica is relative to each other. Bigtable reports a replica's freshness with its *replication watermark*, a data structure mapping its row ranges to the range's replication position. A *replication position* describes, for a given row, the earliest *replication timepoint* that has not been received from another replica (also called a *replication source*).

When a user applies a mutation to a row on a replication source, the source's tablet server tags the mutation with its local logical clock as the mutation's *replication timepoint*. The replication timepoints of mutations applied on the same row on the same replication source are monotonically increasing in the order the mutations are applied, since a row is hosted on at most one tablet server at a time, and a tablet cannot be loaded on a server with a lower logical clock than the replication timepoints it had seen.

Mutations on the same source and same row are also replicated to other replicas in replication timepoint order, so a replication timepoint is sufficient for a replica to represent its replication progress for one row on a source. However, there are no ordering guarantees across rows or sources, so a replica must track replication progress

from multiple sources and row ranges individually. For computational tractability and efficiency of frequent updates, the replication position is tracked as a map from source to the interval map (user row range $\rightarrow$ replication timepoint).

As an example, Figure 1 shows the replication watermark of one replica, R3. There are two other replicas in the replication configuration (R1 and R2), so the replication watermark contains three interval maps, one for each source R1, R2, and R3. R3's row range is split into 3 tablets, covering row ranges $[0, a)$, $[a, e)$, and $[e, \text{inf})$ respectively. Each tablet tracks its own row range's replication position. For example, the row range $[a, e)$ is covered by four different tablets on R1: one covering $[a, b)$, $[b, c)$, $[c, d)$, and $[d, e)$ respectively. Because each of R1's tablet propagates their changes to R3 independently, the replication timepoint of some row ranges (e.g., $[c, d)$) can be ahead of others (e.g., $[d, e)$).

Recall that: (1) different row ranges in a table may be hosted on different tablet servers, (2) row ranges are split between different tablets in each replica, (3) different tablets are hosted on different tablet servers, and (4) tablet servers have varying network and load characteristics. Therefore, the rate at which two replicas can mirror data to each other may differ from tablet to tablet. As a result, for different rows in a replica, the replication timepoints of sources in that row can be very different.

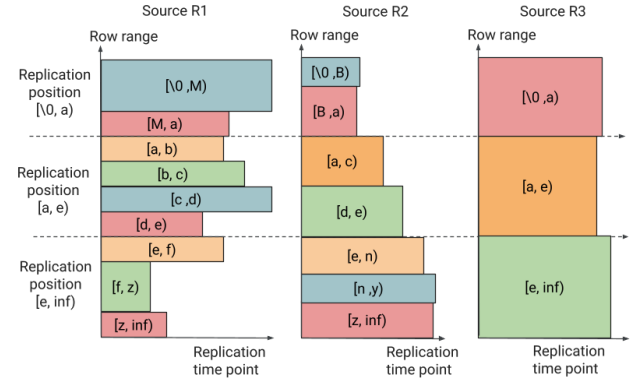


Figure 1: A replication watermark. Note that different sources may be more up-to-date on different row ranges.

3.2 Consistency Model

Bigtable's eventual consistency model guarantees that, in a given replica's replication watermark, the replication timepoint of a given source in a given row does not go backwards. It also guarantees that mutations from *different* sources on the same row are globally sequenced. (Most of the time, the sequencer is just the replication timepoint, plus a tie-breaker based on the source's ID.) Finally, it guarantees that the effects of all mutations under the replica's replication watermark, applied in sequencer order, are visible to readers of the replica (i.e., conflicts are resolved with last-write-wins strategy). The net effect is that, once a mutation's effect is visible on a replica, that effect remains visible until the replica's replication watermark advances to include a mutation with a higher sequencer that applies on the same row. Note that this consistency model gives

no guarantees about consistency across rows, or across replicas, i.e., ACID guarantees only apply to single rows. However, this model has proven to be surprisingly versatile in various scenarios. Common use cases within Google include the following:

Isolating high-throughput traffic from low-latency ones. Often, Bigtable is the backend database for user-facing operations that require low-latency operations (e.g., shopping data), but that same data is then used as inputs to batch workflows that don't require up-to-date data (e.g., analytic pipelines). A common replication use case is for a primary replica to handle low-latency traffic, while a secondary replica is used to handle high-throughput traffic. The different replicas can have different cache sizes (Section 8.2) or be provisioned on Bigtable clusters optimized for their service type. If a team doesn't use replication, they would have to rely on Bigtable features such as offline access (Section 8.3) or request priority (Section 6) to achieve the similar goals.

Multiple primaries. Many applications do not require ACID properties across rows. For example, an application can keep each user's data in a single row if it does not require consistent read/write across different users' data. As long as the application accesses the same replica for each operation in a user session, and for the application to pick a replica that contains the updates for that user from the last session, the per-row per-replica ACID guarantee is often sufficient for user-visible application effects. User data is frequently accessed from a single geographic location, allowing for fast reads from and writes to a local replica, with writes being replicated to other clusters for redundancy.

Failovers. Bigtable is also often used as intermediary storage of long-running pipelines. In a replication set up, a secondary replica can act as a standby table, to take over if the first one fails. When the primary fails, the pipeline can be restarted after replaying data written after the standby's replication watermark.

Bigtable's eventual consistency model leads to lower cost, higher throughput, lower latency, and only requires a single replica for writes. For these advantages, Bigtable replication appeals to teams that can work around the limitations of eventual consistency.

3.3 Replication Configuration

Replicas are configured to mirror data with each other in *replication configurations*, which record participating replicas' locations, column families, as well as other information. The replication configuration is stored in Chubby [14], and is updated by Bigtable masters hosting its replicas. The Bigtable masters cooperate between themselves to ensure a replication configuration is updated consistently. The replicas also copy relevant details from the replication configuration into their own table metadata.

A tablet server will watch the replication configuration of replicas it has loaded tablets from, so that updates to the replication configuration are propagated asynchronously, but within single-digit seconds, to tablet servers. The tablet server then executes the actual mirroring of data (Sections 3.4).

Bringing up a new replica in an existing replication configuration requires populating the new replica with existing data from other replicas. A key challenge is copying over the existing SSTables to the new replica while continuing to allow in-flight mutations to the cluster. Bigtable addresses this with a process internally known

as Bigtable Efficient Add Replica (BEAR). The core mechanism of BEAR is to allow the bulk copying of SSTables while concurrently processing new mutations, ensuring that newer updates correctly overwrite older data in the copied SSTables. To achieve this, BEAR assigns the copied SSTables an artificially low sequencer. As a result, even if a copied SSTable arrives at the new replica after an "in-flight" mutation mapping to the same row, the in-flight mutation's higher sequencer prevails during conflict resolution. This design seamlessly integrates the boot-strapping of a new replica with the live replication traffic without requiring any write downtime.

3.4 Replicating Mutations

Bigtable replication adopts a *pulling model*: destination replicas pull remote mutations from source replicas where original mutations happen. Compared to a pushing model, pulling reduces the overhead on the source replicas where writes happen, it also simplifies state tracking and recovery when pulling tablet servers crash.

When a tablet server receives a mutation to a replica, it tags the mutation with a replication timepoint, and stores the mutation to the *replication log*, a system locality group hidden from the user. The replication log is used to serve pull requests from other replicas. Like user data, replication log entries are stored in key-value format. The entry's value is the serialized mutation itself, while the key is a string-encoded logical tuple of replication source, timepoint, and (user) row. This key ordering optimizes serving of pull requests from other replicas. During minor compactions, replication log entries end up in dedicated SSTables. The key order allows the most recent mutations to be served from memory, while older ones require typically only one sequential read off a single SSTable.

A replication log entry is no longer useful after all other replicas have pulled and applied its mutations. Tablet servers periodically perform replication log garbage collection, known as discard compaction, where it checks the replication position of other replicas, and prunes entire replication log SSTables without useful entries.

The *puller protocol* guarantees exactly once delivery of remote mutations. This is important for non-idempotent mutations such as those for aggregate types (Section 4.3) where a delta instead of a final value may be encoded. To achieve exactly once delivery, each tablet tracks its own replication position, and requests only replication log entries that it hadn't seen. Recall that the replication position is a map from source to the interval map (user row range $\rightarrow$ replication timepoint). This map is loaded into memory at tablet load time and persisted during minor compactions and tablet unloads. A tablet potentially pulls from multiple remote tablets, even those belonging to the same source, because tablet boundaries will likely not perfectly align across replicas. For each remote tablet, the destination tablet queries its in-memory replication position, computes a two-dimensional (row range, replication timepoint) map of interest, and sends a pull request. The remote tablet then reads its replication log and returns any found mutations that fall in the requested map of interest. Upon receipt of these mutations, the destination tablet then applies each mutation and updates its replication position. If no mutation is immediately available to return, the request is queued until a mutation happens or the request's deadline is exceeded. As an optimization for monitoring and observability, we introduce periodic dummy mutations called *heartbeats*

to serve responses for queued pulling requests. Heartbeats happen around every minute at all replicas. This provides a way to ensure no pull request is forever stuck due to lack of user generated mutations. Consequently, replication watermarks can steadily progress in the absence of user mutations.

3.5 Automatic Garbage Collection

Bigtable supports automatic garbage collection (GC) of values based on their age or number of versions, or both (e.g., $\text{age} > 1\text{week}$ or $\text{versions} > 1$). For an unreplicated table, this is simply done during major compactions by not including the GC-ed versions in the resulting SSTables. For a table in a replication configuration, the GC may race with user deletions on other replicas. To avoid divergence, replicas perform GC by emitting delete mutations, which get assigned a sequencer and are replicated to all other replicas just like normal user mutations. However, replicating GC deletions poses another challenge with 3 or more replicas. For a motivating example, consider 3 replicas A, B, and C, each with version v1 and a GC policy to keep just 1 version. After a user writes v2 on A and replica B pulls v2, the GC runs on B: if GC was allowed to delete v1 on B before C saw v2, then C could pull v1 deletion from B before v2 from A, and thus temporarily end up with no versions at all, which violates Bigtable's consistency guarantee, because at no point on any replica is v1 deleted before v2. As a result, GC must count only versions that were replicated to all other replicas. This does not affect age-based GC.

3.6 Replication Latency

The *replication latency* of a mutation is measured as the time between the apply time at the original replica and that at another replica. In practice, most data have replication latency of under a minute. Bigtable replication by default runs at best effort and works off spare resources on servers. It is scheduled by the tablet server at the lowest priority and is deprioritized compared with user requests. It is also subject to various throttling based on server CPU and RAM usage to ensure replication does not overload servers or cause out of memory (OOM) errors. Bigtable users have a wide range of consistency requirements and consequently have different tolerance of replication latency. Some users accept high replication latency, for example, when replicating to and from replicas for batch workloads. They keep replication priority at the default lowest level and thus save on compute and network costs. Other users favor near-real-time replication latency. They increase replication priority and relax throttling constraints in order to allow replication work to compete more fairly with user requests.

4 In-database Processing

Historically, Bigtable has served as the primary ingestion and storage layer for large-scale applications. Bigtable deployments leveraged the MapReduce [22] framework for large-scale parallel computations, utilizing specialized integration wrappers that facilitated Bigtable's role as both an input source and an output sink. Under this paradigm, data processing workflows were offloaded to external distributed execution engines such as Flume [16], Apache Beam [1],

Apache Hadoop, and Apache Spark. Recently, Bigtable's architecture has evolved to incorporate features that support pushing down computations directly to the database engine.

4.1 Queryability

Bigtable frequently serves as a high-throughput ingestion layer and processing environment where raw data is transformed into structured formats for downstream analytics. A common pattern involves storing nested data structures—such as JSON, Protobuf, and Avro—which necessitates granular attribute access during query execution. To address these requirements, SQL was integrated as the primary interface, leveraging its expressive power and industry-standardization. To maintain the flexibility of the Bigtable data model, GoogleSQL [28]—a SQL dialect used in many query engines and services at Google—was extended with support for collection types (maps and sets) and temporal filters [12] to support point-in-time queries (as_of operator) and historical values leveraging cell timestamps. Furthermore, structural transformations, including pivoting and unnesting, were implemented to accommodate the three-dimensional wide-column architecture [9]. The system now also supports Protobuf descriptors, enabling the treatment of serialized objects like user-defined types within the SQL framework. Historically, users implemented hierarchical structures within the linear key space using delimited composite row keys. The introduction of row key schemas [11] formalizes this implicit structure by mapping opaque byte-strings to strongly typed SQL columns. This formalization allows the execution engine to natively parse key components, facilitating optimized range scans and predicate pushdown on individual segments through standard SQL syntax.

4.2 Change Data Capture

Change data capture (CDC) is a database technology that allows users to receive all mutations made to their table in real-time, a crucial feature for use cases such as real-time analytics, data synchronization/replication, and fraud detection. CDC streams are sharded at tablet boundaries to ensure scalability. CDC consumers must manage state hand-offs when tablets split and merge or the replication topology changes to ensure changes are delivered correctly with at-least-once guarantees. To support CDC consumers in managing these hand-offs, each CDC stream tracks its *position* across three dimensions: rows, replicas, and logical time. Each update from the stream is accompanied by the updated position, a.k.a. the continuation token, which consumers combine or split to specify the positions to resume processing with new streams along updated tablet boundaries.

To shield users from this complexity, we've developed a CDC Beam Adapter [2]: a user provides a function to execute on each change and the adapter running on Cloud Dataflow [27] guarantees at least once execution of each change. The adapter is structured with a control plane and data plane. The control plane ensures at most one worker is streaming from a tablet. The workers consume the CDC stream and execute the user-provided function. For durability and efficiency, the control plane uses a second Bigtable, similar to Bigtable's metadata table, to track worker assignments for tablets to process and store continuation tokens.

4.3 Counters and CRDTs

Counters and other conflict-free replicated data types (CRDT) [42] are a popular mechanism for scalable metrics collection for analytics pipelines. Systems like Redis [24] and Cassandra [33] are popular partly because they provide a lightweight scalable solution for these types. Without specialized CRDT support for these cases, users are typically forced to use the following awkward alternatives: (1) Expensive, low scalability, generic read-modify-write (RMW) transactions to perform an update on the existing value each time. While supported for single-cluster tables, this approach is incompatible with Bigtable's asynchronous replication, which is designed to transmit individual cell value updates rather than coordinated cross-cell operations. (2) Explicitly store each delta separately in the table and then either perform the aggregation on each read, or use a complicated, externally managed background collector task for reducing fully replicated delta values to a single summary statistic.

CRDTs are most efficient and simplest to build when they're only required to support commutative operations (e.g. increments and decrements to a counter). In cases where only commutative operations are allowed, it's sufficient to keep running totals at each replica and directly apply the operations as each replica receives them. This design is insufficient, however, when the CRDT must also support noncommutative operations, such as deleting (i.e., clearing) a counter, which would create inconsistencies like the example in Figure 2. For business needs, we require a design that handles deletes. While other systems like Redis and Cassandra often have limitations or complex procedures for handling deletes on their counters [15, 38], deleting Bigtable CRDT data is as simple as deleting any other type of data.

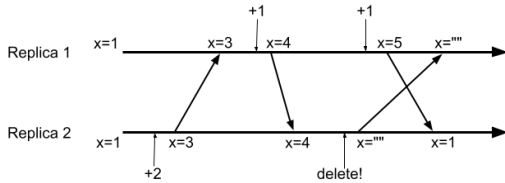


Figure 2: An example of inconsistency resulting from applying delete operations directly to a running total as they arrived at each replica. The two replicas disagree due to the noncommutative behavior of a delete.

To support both commutative increments and noncommutative deletes across replicas, Bigtable must track both the current total and enough history to correctly update the total in the face of asynchronously delivered globally sequenced updates. Our CRDT implementation is specifically designed to fit into the existing LSM tree storage, each layer of the LSM tree stores a subtotal in a structure called a *changelog*. Changelogs do not physically store every operation ever performed, however. We structure it as two parts: (1) A *resolved* subtotal value for the operations that cannot overlap in sequencer order with another layer of the LSM tree. (2) A log of *unresolved* operations for which another layer may have operations that would be sequenced between the resolved operations and the unresolved operation.

Unresolved operations correspond to: (a) *Recent updates* which are sequenced in the 'future' relative to what this replica has pulled so far from all replicas. These are values whose sequencer is greater than the low water-mark (LWM) at the time when this layer was created and a new layer is started above. (b) *Late arrivals* which are sequenced in the 'past' and were pulled after lower layers of the LSM tree were already created. These are values whose sequencer is less than the high water-mark (HWM) value at the time when layers below in the LSM tree were created.

As an example Figure 3 shows a changelog structure for a single LSM layer. In this example, the replication HWM recorded *before* this layer was 10, and the LWM *after* the layer was finalized was 20. It has: (1) A resolved subtotal of 10 for operations between sequencers 10 and 20. (2) Three recent updates: +6 at sequencer 22, +5 at 23, and +1 at 24. (3) Two late arrivals: +1 at 4 and +3 at 5.

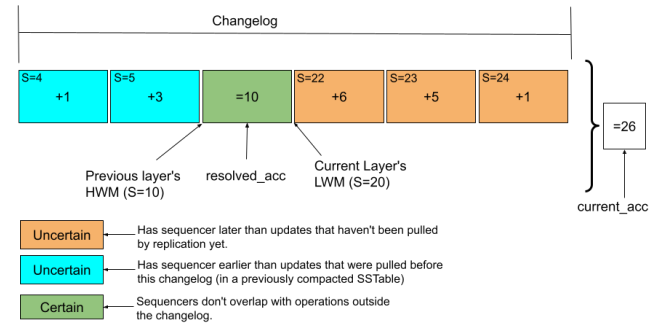


Figure 3: A changelog structure, which holds individual operations which may overlap in sequencer order with values in other parts of the LSM tree.

When Bigtable merge compacts the LSM tree, the logs of the merged layers are combined to make a new changelog, *trimming* the individual operation entries now resolved in the new changelog, incorporating them into the resolved subtotal. See Figure 4. When deletes are processed by the tablet server, operations in the memtable's changelog sequenced before the delete are removed directly. Later, during changelog trimming during later merges, the merged changelog also drops operations which were sequenced before the most recent delete observed. Changelogs and changelog trimming during compactions have been a critical part of making aggregates correct without sacrificing performance, by keeping values as small as possible without sacrificing consistency.

Aggregates have become an extremely popular feature among Bigtable customers. Use cases range across a wide variety of scenarios such as IoT-fleet data collection, social media and ad network impressions tracking, and anomaly detection for network services.

4.4 Materialized Views

Bigtable has historically served as the foundational storage for complex data processing systems. These pipelines often derive secondary data products from a primary source—ranging from simple secondary indices to complex analytical rollups. Traditionally, Google engineers constructed these pipelines using external batch systems like MapReduce [22] or Flume [16]. More recently, the

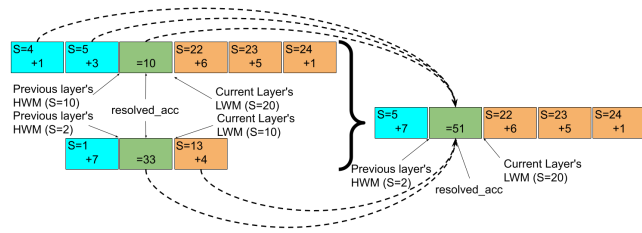


Figure 4: A changelog merge operation, performed during merge compactions.

advent of CDC (Section 4.2) has allowed users to move toward streaming architectures using external engines like Dataflow [1]. However, maintaining these external pipelines introduces significant architectural complexity and operational "headaches," such as managing separate resource quotas, handling upgrades, and ensuring end-to-end consistency across disconnected systems.

To eliminate this overhead, we introduced native *Materialized Views*. This feature allows users to define a transformation as a standard GoogleSQL query (Section 4.1), which Bigtable then executes continuously as data arrives. By internalizing the derivation logic, we can leverage Bigtable's existing consistency models and reduce the user's management surface to a single SQL statement.

Our implementation utilizes the *coprocessor* framework [21]—an internal execution layer that allows Bigtable to run logic directly on tablet servers—to orchestrate a server-side MapReduce-style pipeline. The process is triggered at the storage layer: as updates are applied to the source table, the server records the local logical clock value in a hidden column for each row. A *mapper* coprocessor is then signaled to process these updates and propagate them to a hidden *shuffle table*. Because materialized views often require re-grouping data by a new primary key, this shuffle table acts as an intermediate aggregation point. A downstream *reducer* coprocessor aggregates these shards before a *finalizer* encodes the results into the output table (see Figure 5).

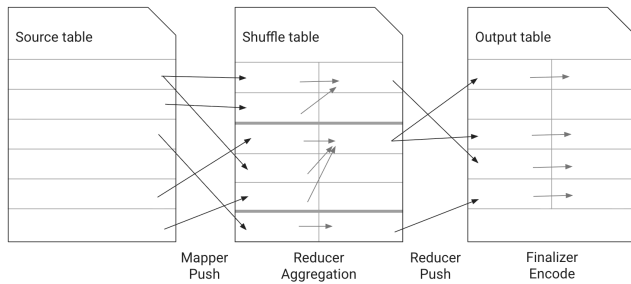


Figure 5: The MapReduce-style pipeline used for Bigtable materialized views.

To track progress and processing delay, the system employs watermarks that track the logical clock value associated with each batch of updates. Each stage of the internal pipeline maintains its own watermark to indicate the currency of the data it has processed. This design ensures that the materialized view remains

eventually consistent with the source table and provides a strictly monotonic progression: updates from a particular source row are never processed out of order, and updates are never "lost" once applied, similar to the consistency provided by replication. Note that each copy of a replicated table maintains its own materialized views, with view propagation occurring strictly after replication.

Customers are using materialized views across a variety of domains and use cases. As a few examples, music streaming services are using them to build dashboards to track engagement on different device types, online retail services use them to build up business intelligence dashboards to improve recommendations, and automotive manufacturers use them to quickly identify trends in vehicle maintenance.

5 Scalability

Bigtable was designed to provide a large-scale database system for managing Google's data. As data sizes grew, we made many changes to accommodate the growth. This section presents some of the scalability challenges we faced and the solutions we devised.

5.1 Horizontal Scaling

Bigtable was designed for horizontal scalability. Tables have no size limit other than local capacity limits. As tables grow or the load increases, more tablets are created by automatic splitting based on size and more servers are added to the Bigtable cluster. The tablets are distributed to all eligible servers to spread the load. However, too many tablets leads to increased management overhead, and in turn to poor performance. Each tablet consumes some memory and CPU for its management, and when tens of thousands are loaded on a server, the aggregate overhead for managing all of them can consume significant CPU and memory, which could be otherwise used for serving. Clients are impacted as well, as they need to locate more tablets, requiring a larger location cache. The rebalancer (Section 5.7) has more work, potentially becoming less responsive to load changes. The load on metadata grows, as well as the Bigtable master.

Bigtable has a small default disk split size, so it works out of the box for smaller tables, allowing them to spread across many tablet servers for load balancing and avoiding hot spots. Left unchanged for larger tables, it leads to an explosion of tablets. Bigtable performs best with fewer tablets (but still several thousands per server), and that is achieved at scale by making cold(er) tablets larger and hot(er) ones smaller. The disk-split size is large, but tablets split and rebalance quickly when under load. Note that increasing tablet size alone is not enough, since tablets cannot get too large without other penalties such as increased duration for compactions. Other improvements have been made to increase the ability of the master and tablet servers to handle the inevitable increase in tablet count, explored in more detail below.

Bigtable only provides single-row transactions. As such, data that need to be updated atomically have to reside on the same row, sometimes resulting in large rows. Excessively large rows often have performance challenges, and we advise the application to avoid that, by say revising the application logic.

5.2 Vertical Scaling

Several changes have been made to increase the ability of a tablet server to handle more tablets. Making individual tablet maintenance cheaper for the server was necessary. Dropping the reference on commit logs for tablets without updates (also called *read-only tablets*) after a predefined period of time cuts tablet unload and recovery time as well as resources needed to perform these operations. Discard compaction (for replication logs) is a technique that deletes replication log SSTables that are below the low replication watermark, without actually reading the data and making decisions about what to discard, as the latter is more expensive. Moving the expensive portion of compactions, file building, outside of the tablet servers frees up CPU and RAM that can be used for handling more tablets and more user-facing traffic. Improving the overall scheduling of background operations and the way they interact with each other (relative priorities, fewer preemptions of expensive operations) helps free resources as well.

As larger machines became available, Bigtable has adapted by deploying larger servers (more RAM and CPU) in production. Larger servers have several advantages. Examples include better resource utilization, higher per-server throughput, better handling of occasional large rows, better at absorbing traffic spikes, and the ability to manage a larger set of tablets. Some changes were needed to take advantage of larger servers. Commit logs needed sharding to maintain (or even improve) the write throughput, because a single file's throughput is limited, so more files are needed to write to in parallel from a tablet server (but too many files decrease batching and throughput as a result). They are also grouped in ways that expose fewer tablets to recovery delays, thereby enabling faster recovery.

5.3 External Compaction

Compactions are bursty operations that take up much CPU and RAM, and can affect other operations on the same tablet server. For this reason they were “externalized” as follows: compactions are initiated and coordinated in the tablet servers with the most expensive part, file building, happening outside of the tablet servers. A separate set of jobs handle file building - merging a set of input files into one or several output files at the tablet server's request. This approach is used only for merge and major compactions; minor compactions read data from memory, and happen in the tablet servers. The file building job is autoscaled (independently from tablet server autoscaling, see Section 8.1) to accommodate fluctuating compaction loads. This approach allows compacting more tablets in parallel, and may even speed up compacting individual tablets (because it allows for more parallelism at the locality group level). It also allows tablet servers to spend most of their resources on serving user traffic. This is a win for isolation as well as for resource efficiency (as an example of resource tiering discussed in Section 8.3), since the file builder jobs can use cheaper resources.

5.4 Master Scalability

The Bigtable master is responsible for assigning tablets to servers, among many other things. It's a single process, acting as the control plane for the Bigtable cluster, and not part of the data path. We have employed many techniques for the master's scalability, with two

highlighted here: (1) Fine-grained locking. The orders of magnitude increase in tablets did cause lock contention within the master and required changes such as using fine-grained locking for certain data structures that manipulate tablets, to allow higher concurrent updates. (2) Parallel schema updates. The original implementation performed table schema updates serially. Over time, as schemas became more complex, schema updates got slower. This was exacerbated as Bigtable clusters grew larger and schema changes became more frequent. We address this challenge by performing table schema updates in parallel.

5.5 Parallelizing File Garbage Collection

As described in the 2006 paper [17], Bigtable uses a mark-and-sweep approach for garbage collecting files. The entire process - identifying obsolete SSTables and commit logs and deleting them - used to be executed by the Bigtable master, serially. That worked as long as the Bigtable clusters remained relatively small. To support larger clusters with many more tablets and files, we moved the work to tablet servers and heavily parallelized each stage. The master continues to coordinate the process by dividing the work and assigning it to tablet servers, keeping track of progress and collecting stats. The more expensive parts, such as reading metadata tables, writing intermediate data in a dedicated system table, issuing requests to the file system for the list of files and to delete them, are done by tablet servers that serve the data operated upon - that is, the metadata and the system table storing intermediate state.

This approach scales well, resulting in a very fast pass, which is important for reclaiming storage and staying under disk quota. One round (a.k.a. pass) of identifying obsolete files and deleting them takes a few minutes to tens of minutes, depending on the size of the Bigtable cluster. This includes files belonging to live tables, copy-on-write snapshots and remote snapshots (tables copied, at the file level, from a remote location).

5.6 Tablet Locator Scalability

Bigtable clients locate data by reading tablet metadata maintained by the servers. This metadata is stored in a table, with tablets either co-located with the user tablets, or isolated on dedicated servers. The client maintains a location cache to avoid reading the metadata too often. More clients and larger jobs, with a fraction of them short-lived and not taking advantage of the client cache, increased the load on the metadata, causing frequent hot-spotting. Several solutions were put in place to divert location traffic away from the metadata table. We introduced location “hints” - the old location of a tablet (a tablet server) maintains a small cache of locations of recently unloaded tablets. When a tablet server gets contacted by the clients for tablets it has hints for, it sends the last known location, which the client uses, avoiding a metadata read. If the new location is not correct either, the client library issues a metadata read. Since the hints are not enough, and don't work for new jobs with cold location caches, we have offloaded the location lookups from the metadata servers to a new group of servers called *location cache proxies*. These jobs, like the file building ones for compactions, are independently autoscaled to accommodate fluctuations in traffic.

5.7 Rebalancer

The rebalancer is the component in the master that is responsible for evenly distributing load across tablet servers. It uses metrics reported by the tablet servers, such as CPU load, RAM utilization, RPC queue length, smoothed over a period of time, to identify heavily loaded tablet servers. It instructs them to shed load, with the purpose of reducing CPU usage and RAM utilization by a specific amount. The tablet server chooses which tablets to unload to meet the goal. Once unloaded, the master decides where else to place them, typically on the most lightly loaded tablet servers. In this manner, potentially over multiple rounds of rebalancing, the load will then be gradually smoothed out across the tablet servers. The rebalancer will not try to shift too much load in a single round because load is frequently short-lived and moving tablets incurs a non-negligible cost. We have observed that picking the hottest tablets for unload is not always the best option. Instead, keeping them in place and picking cooler candidates for unloading is more preferable because doing so avoids an increase in location lookups, which come from all the clients who now have the wrong location of the hot (but moved) tablet. In extreme cases, only one hot tablet may remain on a tablet server. Hotspotting like this is an anti-pattern and we encourage Bigtable users to find alternative ways to shard their data to avoid hotspotting. Furthermore, we have developed a visualization tool to show us the hotness of each key, which has been very useful in identifying hotspots.

6 Performance

Bigtable stores its data in a log-structured merge tree (LSM-tree) [36] backed by immutable SSTables. LSM-trees are optimized for write performance. However, reads can traverse several SSTables to access all data in the tree, this can be costly. Bigtable has introduced and enhanced several key algorithms on the read path to ensure optimal performance which we outline below.

6.1 Row Cache

Bigtable’s low-level *block cache* holds SSTable data blocks which reduces I/O costs associated with block retrieval from Colossus. The cache operates at the same abstraction layer as disk access, so request processing requires block-centric traversal of the in-memory data structures to build a full-stack row result from the SSTable stack. Bigtable’s new *row cache* builds on the block cache to cache data at row granularity. This reduces CPU usage by up to 25% for point read operations and alleviates hotspotting problems for frequently accessed data. In contrast to cached block storage, cached rows use a sparse representation of the block data, maintaining only the data accessed by a query within any row. This format allows queries to reuse row-cache data so long as the cached row contains the required data. In the event that a new query requires data that is not present in the cache entry, the request falls back to block reads and populates the row cache structure with the missing data. Both row and block caches share the same memory pool and employ an eviction algorithm to optimize performance across a diverse set of query types, balancing block caching for breadth of response versus row caching for high-throughput access to the most frequently accessed data. Row caching also considers row size in its storage optimization algorithm to maintain high hit rates.

6.2 Bloom Filters

A Bloom filter is a space-efficient probabilistic data structure that can tell whether an item is in a set, it has a small number of false positives (item may be in the set), but no false negatives (item is definitely not in the set). In Bigtable’s case, Bloom filters reduce the search area to a subset of SSTables that may contain data for a given row-key, reducing costly disk access.

Bigtable, like many database systems, originally stored the entire Bloom filter into DRAM for fast access. However, this resulted in high memory usage in some workloads overloading the servers. To mitigate this, we introduced the concept of *sharded Bloom filters* where we maintain a separate Bloom filter for a subset of the key range. These filters are treated as a special piece of data using the same indexing and caching system that we use for data blocks. This resulted in storing only the relevant Bloom filters in DRAM which was an order of magnitude less space than the entire filter.

We observed that Bloom filters were utilized less than expected. This was due to our implementation expecting both the “column family” and the “column” in the read filter, while a high percentage of customers filter by “column family” only — which means the Bloom filter can’t be used. We increased utilization by implementing a *hybrid Bloom filter* that was applicable in both cases, resulting in a 4x increase in utilization. While this change made the Bloom filters larger, the overall disk footprint increased by only a fraction of a percent, as Bloom filters are typically two orders of magnitude smaller than the data they represent. The CPU cost of accessing the Bloom filters was also high, so we made enhancements to optimize runtime performance: (1) Local cache for individual reads. When queries select multiple column families and columns in a single row, it is common that the query will use the same Bloom filter. We take advantage of this by storing a local cache of the Bloom filters used for the query being executed. (2) Bloom filter index cache. Since Bloom filters are stored as data, accessing them for the first time involves fetching three blocks — two index blocks and a data block — then performing a binary search on all three. To avoid this overhead we built a custom in-memory index for just the Bloom filters. This cache tracks which Bloom filters we have in our block cache and provides direct access to them. (3) Adaptiveness. Even though Bloom filters are relatively cheap to access, they are not free. Many workloads look for data that is always present in the dataset, which means the Bloom filter lookup is wasteful. To improve these scenarios the filters periodically evaluate their effectiveness and can turn off if that is the best for performance.

6.3 Request Priorities

In addition to internal database performance enhancements, we’ve added user-driven throughput improvements for point operations. Users can annotate their most important traffic with a scheduling priority, which can bring higher throughput and lower latency to that traffic versus lower priority traffic. We built this feature to improve Bigtable’s support for hybrid transactional/analytical processing (HTAP) workloads by improving isolation between transactional operations, typically single row reads, and analytical operations, which are complex queries with multi-row results.

7 Reliability

Data integrity is of paramount importance in a storage system - the user should always retrieve the data that was successfully committed. In practice data integrity can be impacted by a variety of reasons, such as temporary or permanent hardware malfunction or software bugs that cause memory to be modified unintentionally. At the scale Bigtable operates, any conceivable hardware/software error might happen. While difficult to prevent altogether, it's possible to reduce the probability of these events happening to the point where it's a very rare occurrence. Bigtable has achieved this by employing a variety of techniques, some of them described below. Notably, the extensive use of checksums, while expensive in terms of resources, is paying off.

7.1 Snapshots and Backups

Bigtable has copy-on-write snapshots, which makes them very cheap to create. They get more expensive as the source table moves on, accepting writes, compacting tablets, and creating new files for the data. Snapshots protect against some issues that do not impact files shared with the source table. They protect against bugs and accidental data deletion. Backups start as snapshots, with a background process copying the underlying files to a backup service, preferably in another cluster. They offer disaster recovery.

7.2 End-to-End Checksumming

Checksums are the primary tool to protect data integrity. We use CRC32 checksums at various data granularity levels. Row keys, column keys and values are checksummed. Mutations (data structure incorporating a set of updates) are checksummed, and each file has a file-level checksum. In addition to this, Bigtable uses authenticated encryption for data at rest, a form of encryption that ensures confidentiality and integrity of encrypted data. Designed to provide security guarantees against malicious attackers trying to change encrypted data, it also gives us protection for data integrity.

Bigtable implements end-to-end checksumming to protect data integrity from the client to storage and back. Users can provide CRC32C checksums for values at write time. If none are provided, the client library generates them. These checksums are validated in several places in the write path: when the RPC is received by the tablet servers, before writing to the commit log, before updating the memtable, in the replication path. In addition to checksum validation in the tablet server, Bigtable provides checksummed data to the storage layer on write, which provides end-to-end checksumming as well. If errors are detected in the write path, several actions are possible: the operation ends in error or is retried from the client, or if the server detecting the error is suspected of experiencing hardware issues, crashing the server to avoid further errors. The row, column and value checksums are persisted in the SSTables next to the data. They are checked on the read path, in the tablet server and client-side.

7.3 Continuous Checks

Bigtable compactions rewrite all the data regularly. This is necessary for garbage collection, to apply deletion markers, and to rotate encryption keys. The checksums persisted in SSTables are validated during compactions, and if errors are detected the compaction fails,

and the tablet is taken offline and “quarantined.” Once that happens, the system sends alerts and the SREs will take action. Possibly the most successful but resource consuming integrity preservation mechanism Bigtable has implemented is verifying the integrity of SSTables after they’re built but before they are “installed” (the process of recording the files in metadata, making them part of the system). The SSTables are read back, decrypted, uncompressed, and each entry is parsed and validated, including all the checksums. This action is performed by the file builder jobs, outside of the tablet servers. While resource intensive, the end result is that virtually no SSTable inconsistencies have been observed in years. This is very important, since SSTable problems can impact the entire tablet. Other checks are designed to detect gaps in commit logs. If detected, the system stops garbage collecting files and sends an alert. More checks are performed on vital metadata while in memory (checksums, integrity checks), and validated before use, or before the metadata is persisted in the metadata table. Encryption keys are checksummed and validated before each use.

8 Resource Efficiency

Our efforts on resource efficiency (using less to do more) can be categorized into three main themes: (1) Reduce headroom. This aims at increasing utilization by reducing over-provisioning of resources. Our primary mechanism is automatic adjustments to the amount of provisioned resources. (2) Reduce usage. This aims at increasing productivity by reducing the total cost of resources used for each unit of work performed. (3) Resource tiering. This involves reducing costs by using cheaper resources wherever appropriate. External compactors, described in Section 5.3, is an example of resource tiering.

8.1 Autosizing and Autoscaling

Traffic to servers in a Bigtable cluster is always fluctuating and most of the time unpredictable. As a result, setting aside a proper amount of resources becomes a challenging problem. To address this problem, we devised a two-component approach: Autosizing plus Autoscaling. The *Autosizer* is a monitoring process that runs in each Bigtable cluster. It continuously monitors the CPU/RAM load on all the tablet servers as well as other health metrics such as tablet availability, and determines whether servers should be added or removed. The autosizer makes sizing decisions based on the total load on the tablet servers collectively, and relies on the rebalancer (Section 5.7) to even out the load in a timely manner. The servers added or removed come from, and go to, a place called the *slack subpartition*, a pool of idle servers. When this pool is empty, or has many servers for sufficiently long, Bigtable triggers Google’s autoscaling system to either request or return resources. *Autoscaling* is a Google-wide system for resource management. Systems like Bigtable tell autoscaling how much resources they need and can configure autoscaling to upscale or downscale by how much. The requested and returned resources come from and go to a resource pool owned by a product area (PA) and shared by all the services in that PA.

We use this two-component auto approach (instead of one) for two reasons. One, fast reaction time. The autosizer runs on a sub-minute frequency, and the pool of servers in the slack subpartition

are within the Bigtable master's control. Therefore, new servers can be quickly added. In contrast, autoscaling provisions resources from a central pool and spins up new servers, a process that can take several minutes. Employing this two-component approach allows us to strike a balance between fast reaction time and resource efficiency. Two, customizable autoscaling metrics. While monitoring CPU/RAM utilization suffices for most autoscaled compute products, our database requires the ability to monitor and autoscale based on additional health metrics (such as tablet availability) on the servers. The Autosizer provides this crucial customization.

This same underlying two-component system powers the Autoscaling feature in Cloud Bigtable [5], allowing cloud customers to automatically adjust the number of nodes based on CPU and storage utilization, thus optimizing for cost and performance. Autoscaling on storage utilization is unique to Cloud Bigtable, since Cloud Bigtable rejects writes when there is too much data stored per node. When users enable Autoscaling, Cloud Bigtable will automatically scale up the node count to satisfy the constraint, guaranteeing that writes won't be rejected unless you are already at your max nodes and also exceeding the storage limit per node.

8.2 Cache right-sizing

As mentioned in Section 6.1, Bigtable uses a block cache. While caching consists of many aspects, we discuss a particular one here: cache right-sizing. That is, choosing a proper size limit for the cache such that it is both performant and resource efficient. Caching algorithms, beyond a certain cache size, often manifest a diminished return (a knee shape hit rate curve). Therefore, increasing the cache size beyond a certain inflection point costs more RAM but gains very little in performance. We want to predict the cache size for this inflection point. However, predicting a good cache size is challenging due to the ever changing workloads and the complexity and sophistication of modern-day caching algorithms. To this end, we have developed a feedback-loop-based component called the *cache sizer*. The cache sizer operates by continuously monitoring several key metrics like the RAM cost associated with cache usage on each tablet server, and the CPU and I/O resources consumed due to cache misses. Based on this data, the cache sizer dynamically adjusts the cache size limit for each server. This adjustment is guided by a Total-Cost-of-Ownership (TCO) model. Concurrently, it continuously monitors and safeguards performance. Through this continuous feedback loop, we are able to achieve both good performance and resource efficiency.

8.3 Tiered Storage, Offline Access, Bulk Import

Tiered storage allows separating a tablet's data based on age: newer, frequently accessed data is stored in a different set of files than the older, rarely accessed data, often in storage medium with vastly different speed and cost, thereby gaining performance and resource efficiency. Furthermore, the disk split size (See Section 5.1) can be specified separately for the "hot" section, resulting in large tablets that have most of their data in the cold section and a smaller portion in the hot section, which is needed for good load balancing and avoiding hot-spotting.

Many batch processing applications read and write large volumes of data to Bigtable. These workloads are typically latency tolerant,

making the use of high-cost, low-latency serving resources inefficient. To provide cost-effective solutions for such use cases, Bigtable offers two features: *offline access* (for reads) and *bulk import* (for writes), enabling users to leverage cheaper, lower-tier resources.

Offline access works by letting batch processing workers (e.g., Flume [16] workers or Dataflow [1] workers) read data directly from the underlying SSTable files in Colossus [26], mostly bypassing the Bigtable servers. This shifts the compute cost from the tablet servers to the batch workers, which typically run on lower-cost, throughput-oriented CPU tiers. While workers occasionally consult tablet servers for metadata, this incurs minimal overhead.

Cloud Bigtable's Data Boost feature [6] builds upon this principle, offering serverless, on-demand compute for offline reads, completely isolated from the customer's serving cluster. This is done by using Offline Access to bypass the cluster's tablet servers, instead placing the compute onto a multi-tenant pool of Cloud Bigtable's frontend servers. Quota enforcement and billing is done by metering the compute of every request and converting it into an abstract billing unit called a SPU (Serverless Processing Unit, equivalent to 1/1000 of a node's performance).

Analogously, bulk import is used when we need to ingest a vast amount of data from another storage system into Bigtable. Instead of issuing write requests to Bigtable servers, bulk import allows the batch workers to build the underlying SSTables directly from the source data. At the end, they ask tablet servers to update the Bigtable metadata so that Bigtable can recognize and serve the newly built SSTables.

9 Operational Experience

Bigtable was originally intended to be run by a user (or a group of users). However, it quickly became clear that running Bigtable by each team led to fragmented knowledge and ad-hoc solutions. Consequently, Bigtable has been run as a company-wide service, owned by the Bigtable SRE team, since mid-2006. At the time of this writing, most of the company's Bigtable usage is run within the service, and only a small number of teams run their own Bigtable clusters. Due to space limitations, we mainly focus on two aspects of our operational experience: monitoring, and configuring Bigtable.

9.1 Monitoring Bigtable

We run multiple *probers* in a cluster to measure the availability and latency for metadata and user partitions. Our metadata prober enumerates the rows in metadata, and performs a read on every row periodically to detect unavailable metadata. Unavailable metadata ranges can impact location lookups and also writes, as we need to update tablet locations in metadata during tablet moves and compactions. We have varying alerts, based on the number of ranges unavailable and the duration of the unavailability, that generate pages which are handled by the oncaller. We also run a blackbox latency prober, which aims to mimic a well behaved Bigtable user. It runs a fixed workload against a dedicated table in every partition in the cluster. It exports read and write latencies to our monitoring system, and we use this data to build read/write latency SLIs. We have configured pages based on latency SLO, which helps to diagnose problems within a partition, across a cluster, or across multiple clusters.

9.2 Configuring Bigtable

Cluster classes, partitions, and subpartitions. We offer two separate classes of Bigtable service clusters: one (called *serve clusters*) for serving workloads, and another (called *mixed clusters*) for batch oriented mixed workloads, as they provide different levels of availability and latency. While all user workloads, along with system metadata, were initially hosted within a single partition, we later realized that isolating metadata into a dedicated *meta-data partition* would cleanly separate user workloads from system operations. Furthermore, we created additional partitions to segregate workloads by the users. Over time, we further introduced the concept of *subpartitions* to provide more granular isolation. We also introduced different request priorities, isolation classes, and features like offline access (Section 8.3), which made it easier to mix batch and serving workloads without any interference.

Resource management. Initially we allowed our users to provision tablet servers with arbitrary RAM and CPU shapes. However, this made the service harder to operate and performance harder to reason about. We decided to standardize our offering and introduced fixed tablet server shapes. This made the performance more predictable and easier to reason about. We also built a convenient way to perform online reshaping of the tablet servers.

Backups by default. By default, we ensured that every table had a backup, which provided insurance against a large class of application errors. These backups would be taken daily and retained for 24 hours. Users do have an option to choose different backup policies, allowing them to control the frequency and retention of Bigtable backups. They are also able to mix backup policies, and keep backups on select storage media at different frequencies.

10 Related Work

Bigtable [17] is one of the earliest non-relational databases. In the last two decades, the non-relational movement has generated numerous new data stores, and at this point, there may be too many of them to list all. See [20] for a survey of these data stores. According to [20], more than 200 of them (and growing) have been reported, and by their data models, non-relational data stores can be roughly categorized into key-value stores [23, 25], wide-column stores [17, 30, 31, 33], document stores [32, 35, 43], and graph stores [13, 41]. Further type include text search engines, array stores, and vector stores [46]. Since Bigtable is a wide-column store, here we highlight a few data stores in that category.

HBase [30] is an open-source non-relational data store modeled after Bigtable and written in Java. Its API and core functionalities are fairly similar to Bigtable. HBase has been adopted by multiple noteworthy enterprises and serves as a testament to the impact and influence of Bigtable. Likewise, Hypertable [31] was yet another Bigtable “clone” and was at some point sponsored by Baidu, but Hypertable has ended development in 2016. Similarly, Cassandra [33] is an open-source non-relational data store originally developed at Facebook. It shares a data model fairly similar to Bigtable with some differences. For example, column families in Cassandra can be nested but not in Bigtable. Cassandra also supports a query language (Cassandra Query Language, or CQL), something that Cloud Bigtable has recently added [10]. Amazon’s DynamoDB [25] is a non-relational cloud database service; in that regard, it is most

analogous to Cloud Bigtable [18]. DynamoDB grew out of Amazon’s experience with Dynamo [23] and SimpleDB [40], where the former is a key-value store with incremental scalability and predictable performance, and the latter provides a table-based data model with ease of administration but severe scale and performance limitations. The DynamoDB paper [25] is also an experience paper summarizing Amazon’s experience with DynamoDB.

The deluge of non-relational data stores has prompted a debate about the pros and cons of SQL vs non-relational systems [44–46]. The pros of non-relational include scalability and performance, yet in our experience, resource efficiency (Section 8) is also a major attraction of Bigtable. The cons of non-relational include (1) lack of data model standard (every non-relational system seems to have its own data model), (2) lack of high-level query languages, and (3) lack of support for ACID transactions. In our experience, (1) only happens when you need to convert data between data stores, which does happen, but not very often. For (2), both Cloud Bigtable [18] and Cassandra [33] have added high-level query languages. For (3), with some properly defined schema, many applications can live with single-row transactions pretty comfortably. It seems fair to say that there are enough use cases on either side to sustain both camps. These pros and cons have motivated the NewSQL movement, where new SQL systems incorporate the best of both worlds of SQL and non-relational [19, 37, 48]. It remains to be seen if NewSQL can match non-relational’s main pros.

Lastly, while non-relational has been around for twenty years or so and many systems have been proposed, there appears to be scarce literature on the experience of running these systems, especially over a long period of time. The DynamoDB paper [25] is closest to the present paper as it also shares the experience of running DynamoDB at Amazon for about 10 years. Elsewhere, Presto [47] documents Facebook’s decade-long experience of running and improving a distributed SQL query engine. We feel it behooves the community to share more experience on long-running systems.

11 Concluding Remarks

It has been a long and exciting journey running Bigtable while continually improving it along the way to meet the growing demands. Bigtable has proven its longevity in terms of its appeal to our users, its coverage of a wide-range of use cases, and also its extensibility to meet new demands, all at the same time retaining its original simple architecture. Our operational experience also shows that Bigtable can be feasibly managed by a relatively small team of SREs with sufficient domain knowledge. Going forward, we will continue to evolve Bigtable to meet new demands, in terms of adding new features, improving its key aspects (performance, scalability, efficiency, and so on), and also making it easier and more robust to manage and operate.

12 Acknowledgments

We dedicate this paper to the memory of Nick Taylor, who served as a Bigtable tech lead for many years. We are thankful to Chris Taylor for providing valuable feedback on earlier drafts of the paper. Over the years, many many people have contributed to Bigtable, perhaps too numerous to list all here. We are grateful to them for their work on Bigtable.

References

- [1] Tyler Akidau, Robert Bradshaw, Craig Chambers, Slava Chernyak, Rafael J. Fernández-Moctezuma, Reuven Lax, Sam McVeety, Daniel Mills, Frances Perry, Eric Schmidt, and Sam Whittle. 2015. The dataflow model: a practical approach to balancing correctness, latency, and cost in massive-scale, unbounded, out-of-order data processing. *Proc. VLDB Endow.* 8, 12 (Aug. 2015), 1792–1803.
- [2] Google Cloud Bigtable. 2005. CDC Dataflow connector. <https://docs.cloud.google.com/bigtable/docs/change-streams-use-dataflow>
- [3] Google Cloud Bigtable. 2021. How BIG is Cloud Bigtable? <https://cloud.google.com/blog/topics/developers-practitioners/how-big-cloud-bigtable>
- [4] Google Cloud Bigtable. 2024. Celebrating 20 years of Bigtable with exciting announcements at Next. <https://cloud.google.com/blog/products/databases/bigtable-enhancements-at-next24>
- [5] Google Cloud Bigtable. 2025. Autoscaling. <https://docs.cloud.google.com/bigtable/docs/autoscaling>
- [6] Google Cloud Bigtable. 2025. Bigtable Data Boost Overview. <https://docs.cloud.google.com/bigtable/docs/data-boost-overview>
- [7] Google Cloud Bigtable. 2025. Change streams overview. <https://docs.cloud.google.com/bigtable/docs/change-streams-overview>
- [8] Google Cloud Bigtable. 2025. Continuous materialized views. <https://cloud.google.com/bigtable/docs/continuous-materialized-views>
- [9] Google Cloud Bigtable. 2025. Introduction to SQL in Bigtable. <https://docs.cloud.google.com/bigtable/docs/introduction-sql>
- [10] Google Cloud Bigtable. 2025. Introduction to SQL in Bigtable. <https://docs.cloud.google.com/bigtable/docs/introduction-sql>
- [11] Google Cloud Bigtable. 2025. Managing row key schemas. <https://docs.cloud.google.com/bigtable/docs/manage-row-key-schemas>
- [12] Google Cloud Bigtable. 2025. Temporal filters in table functions. https://docs.cloud.google.com/bigtable/docs/reference/sql/table-functions-built-in#temporal_filters
- [13] Nathan Bronson, Zach Amsden, George Cabrera, Prasad Chakka, Peter Dimov, Hui Ding, Jack Ferris, Anthony Giardullo, Sachin Kulkarni, Harry Li, Mark Marchukov, Dmitri Petrov, Lovro Puzar, Yee Jiun Song, and Venkat Venkataramani. 2013. TAO: Facebook's Distributed Data Store for the Social Graph. In *2013 USENIX Annual Technical Conference (USENIX ATC 13)*. 49–60.
- [14] Mike Burrows. 2006. The Chubby lock service for loosely-coupled distributed systems. In *Proceedings of the 7th Symposium on Operating Systems Design and Implementation (OSDI '06)*. 335–350.
- [15] Cassandra2. 2025. Counters in Cassandra. <https://cwiki.apache.org/confluence/pages/viewpage.action?pageId=120732499#Counters-Technicallimitations>
- [16] Craig Chambers, Ashish Raniwala, Frances Perry, Stephen Adams, Robert R. Henry, Robert Bradshaw, and Nathan Weizenbaum. 2010. FlumeJava: easy, efficient data-parallel pipelines. In *Proceedings of the 31st ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI '10)*. 363–375.
- [17] Fay Chang, Jeffrey Dean, Sanjay Ghemawat, Wilson C. Hsieh, Deborah A. Wallach, Mike Burrows, Tushar Chandra, Andrew Fikes, and Robert E. Gruber. 2006. Bigtable: A Distributed Storage System for Structured Data. In *Proceedings of the 7th USENIX Symposium on Operating Systems Design and Implementation (OSDI '06)*. USENIX Association, 205–218.
- [18] Google Cloud. 2025. Google Cloud Bigtable. <https://cloud.google.com/bigtable>
- [19] James C. Corbett, Jeffrey Dean, Michael Epstein, Andrew Fikes, Christopher Frost, J. J. Furman, Sanjay Ghemawat, Andrey Gubarev, Christopher Heiser, Peter Hochschild, Wilson Hsieh, Sebastian Kanthak, Eugene Kogan, Hongyi Li, Alexander Lloyd, Sergey Melnik, David Mwaura, David Nagle, Sean Quinlan, Rajesh Rao, Lindsay Rolig, Yasushi Saito, Michal Szymaniak, Christopher Taylor, Ruth Wang, and Dale Woodford. 2012. Spanner: Google's globally-distributed database. In *Proceedings of the 10th USENIX Conference on Operating Systems Design and Implementation (OSDI)*. 251–264.
- [20] Ali Davoudian, Ling Chen, and Mohammad Liu. 2018. A Survey on NoSQL Stores. *Comput. Surveys* 51, 2 (2018), 40:1–40:43.
- [21] Jeffrey Dean. 2009. Designs, Lessons and Advice from Building Large Distributed Systems (invited talk). In *The 3rd ACM SIGOPS International Workshop on Large Scale Distributed Systems and Middleware (LADIS'09)*.
- [22] Jeffrey Dean and Sanjay Ghemawat. 2004. MapReduce: simplified data processing on large clusters. In *Proceedings of the 6th Conference on Symposium on Operating Systems Design & Implementation - Volume 6 (OSDI'04)*. 137–150.
- [23] Giuseppe DeCandia, Deniz Hastorun, Madan Jambani, Gunavardhan Kakulapati, Avinash Lakshman, Alex Pilchin, Swaminathan Sivasubramanian, Peter Voss, and Werner Vogels. 2007. Dynamo: Amazon's highly available key-value store. In *Proceedings of Twenty-First ACM SIGOPS Symposium on Operating Systems Principles (SOSP '07)*. 205–220.
- [24] Dirk Eddebuettel. 2022. A brief introduction to Redis. <https://arxiv.org/pdf/2203.06559>
- [25] Mostafa Elhemali, Niall Gallagher, Nick Gordon, Joseph Idziorek, Richard Krog, Colin Lazier, Erben Mo, Akhilesh Mritunjai, Somasundaram Perianayagam, Tim Rath, Swami Sivasubramanian, James Christopher Sorenson III, Sroaj Sothikul, Doug Terry, and Akshat Vig. 2022. Amazon DynamoDB: A Scalable, Predictably Performant, and Fully Managed NoSQL Database Service. In *2022 USENIX Annual Technical Conference (USENIX ATC 22)*. 1037–1048.
- [26] Google Cloud Platform (GCP). 2005. Colossus. <https://cloud.google.com/blog/products/storage-data-transfer/a-peek-behind-colossus-googles-file-system>
- [27] Google Cloud Platform (GCP). 2005. Dataflow. <https://cloud.google.com/products/dataflow>
- [28] Google LLC. 2024. GoogleSQL Language Reference. <https://github.com/google/google-sql>
- [29] Ilya Grigorik. 2012. SSTable and Log Structured Storage: LevelDB. <https://www.igvita.com/2012/02/06/sstable-and-log-structured-storage-leveldb/>
- [30] HBase. 2025. <https://hbase.org/>
- [31] Hypertable. 2025. <https://hypertable.com/>
- [32] Ram Kesavan, David Gay, Daniel Thevesen, Jimit Shah, and C. Mohan. 2023. Firestore: The NoSQL Serverless Database for the Application Developer. In *IEEE 39th International Conference on Data Engineering (ICDE)*. 3376–3388.
- [33] Avinash Lakshman and Prashant Malik. 2010. Cassandra: A Decentralized Structured Storage System. *ACM SIGOPS Operating Systems Review* 44, 2 (2010), 35–40.
- [34] C. Mohan, Don Haderle, Bruce Lindsay, Hamid Pirahesh, and Peter Schwarz. 1992. ARIES: a transaction recovery method supporting fine-granularity locking and partial rollbacks using write-ahead logging. *ACM Transactions on Database Systems* 17, 1 (March 1992), 94–162.
- [35] MongoDB. 2025. <https://mongodb.org/>
- [36] Patrick O'Neil, Edward Cheng, Dieter Gawlick, and Elizabeth O'Neil. 1996. The log-structured merge-tree (LSM-tree). *Acta Informatica* 33, 4 (June 1996), 351–385.
- [37] Andrew Pavlo and Matthew Aslett. 2016. What's Really New with NewSQL? *SIGMOD Record* 45, 2 (Sept. 2016), 45–55.
- [38] Redis. 2025. Storing counters in Redis. <https://redis.io/glossary/storing-counters-in-redis/>
- [39] Google Scholar. 2025. Bigtable citations. <https://scholar.google.com/scholar?cites=535416719812038974>
- [40] Amazon Web Services. 2025. Amazon SimpleDB. <https://aws.amazon.com/simpledb/>
- [41] Bin Shao, Haixun Wang, and Yatao Li. 2013. Trinity: a distributed graph engine on a memory cloud. In *Proceedings of the 2013 ACM SIGMOD International Conference on Management of Data (SIGMOD '13)*. 505–516.
- [42] Marc Shapiro, Nuno Preguiça, Carlos Baquero, and Marek Zawirski. 2011. Conflict-free replicated data types. In *Proceedings of the 13th International Conference on Stabilization, Safety, and Security of Distributed Systems (SSS'11)*. 386–400.
- [43] Dharma Shukla, Shireesh Thota, Karthik Raman, Madhan Gajendran, Ankur Shah, Sergii Ziuzin, Krishnan Sundaram, Miguel Gonzalez Guajardo, Anna Wawrzyniak, Samer Boshra, Renato Ferreira, Mohamed Nassar, Michael Koltachev, Ji Huang, Sudipta Sengupta, Justin Levandoski, and David Lomet. 2015. Schema-agnostic indexing with Azure DocumentDB. *Proceedings of VLDB Endowment* 8, 12 (Aug. 2015), 1668–1679.
- [44] Michael Stonebraker. 2010. SQL databases v. NoSQL databases. *Commun. ACM* 53, 4 (April 2010), 10–11.
- [45] Michael Stonebraker. 2011. Stonebraker on NoSQL and enterprises. *Commun. ACM* 54, 8 (Aug. 2011), 10–11.
- [46] Michael Stonebraker and Andrew Pavlo. 2024. What Goes Around Comes Around... And Around... *SIGMOD Record* 53, 2 (July 2024), 21–37.
- [47] Yutian Sun, Tim Meehan, Rebecca Schlusell, Wenlei Xie, Masha Basmanova, Orri Erling, Andrii Rosa, Shixuan Fan, Rongrong Zhong, Arun Thirupathi, Nikhil Collooru, Ke Wang, Sameer Agarwal, Arjun Gupta, Dionysios Logothetis, Kostas Xirogiannopoulos, Amit Dutta, Varun Gajjala, Rohit Jain, Ajay Palakuzhy, Prithvi Pandian, Sergey Pershin, Abhisek Saikia, Pranjal Shankhdhar, Neeraj Somanchi, Swapnil Tailor, Jialiang Tan, Sreeni Viswanadha, Zac Wen, Biswapesh Chattopadhyay, Bin Fan, Deepak Majeti, and Aditi Pandit. 2023. Presto: A Decade of SQL Analytics at Meta. *Proc. ACM Manag. Data* 1, 2 (June 2023), 1–25.
- [48] Jeff Swenson, Andy Kimball, Raphael 'kena' Poss, Rebecca Taft, Jay Lim, Adam Storm, Sumeer Bhola, Paul Bulkley-Logston, Pj Tatlow, Rachael Harding, Rafi Shamim, Aditya Maru, and Irfan Sharif. 2025. CockroachDB Serverless: Sub-second Scaling from Zero with Multi-region Cluster Virtualization. In *Companion of the 2025 International Conference on Management of Data (SIGMOD/PODS '25)*. 648–661.
- [49] Wikipedia. 2025. Google Cloud Platform (GCP). https://en.wikipedia.org/wiki/Google_Cloud_Platform